Who Wrote It?

Identifying LLMs from Their Responses

CMPSC 448 - Individual Project
Arjun Gupta | Sole participant and project lead
October 11, 2026

Repository: github.com/ArjunGupta005/llm-authorship-classification

Abstract

This project tests whether response text can identify its generating LLM using a TextCNN and a bidirectional LSTM, with TF-IDF logistic regression as a baseline. A curated LMSYS-Chat-1M sample contains 2,937 first-turn examples from GPT-4, Claude-1, and Llama-2-13B-Chat. Exact normalized prompts, responses, and conversation identifiers are grouped before a training/validation/test split. On 588 held-out examples, response-only CNN and LSTM accuracy reaches 91.33% and 85.20%, respectively. Input-only models perform above the uniform-random expected accuracy of 33.33%, suggesting prompt-selection confounding. Adding prompts does not improve neural test accuracy. Additional experiments examine transfer to writing excluded from training and structural/lexical features. Findings are descriptive and limited to historical versions, one seed, and a small writing test subset.

Problem definition and research questions

The supervised task maps text to one of three generator labels: Claude, GPT, or Llama. Although these are family names, each label represents a single historical model version in this study. The task is generator attribution among known classes, rather than distinguishing human text from AI text or detecting an unseen generator.

Question	Experiment
RQ1: Identify the generator?	Response-only CNN and BiLSTM.
RQ2: Does the prompt help?	Input, output, and combined text for both architectures and the baseline.
RQ3: Transfer across domains?	Exclude writing from training/validation; evaluate writing and non-writing test subsets.
RQ4: Distinguishing characteristics?	Training-only style summaries, a style classifier, and lexical/formatting ablation.

The central interpretation question is whether performance reflects transferable generator patterns, task distribution, or both. This report therefore includes prompt-only controls, duplicate-group checks, and explicit limitations.

1. Dataset curation and provenance

The source is LMSYS-Chat-1M [1, 2], a dataset of real-world LLM interactions collected in 2023. The project uses publicly released records through the Hugging Face access process; it does not generate new responses or use personal Gemini/Claude histories. Model identity comes from source metadata rather than inference from response text.

Project label	Source model	Retained examples
GPT	gpt-4	990
Claude	claude-1	957
Llama	llama-2-13b-chat	990

Collection used seed 42 and source revision `200748d9d3cddcc9d782887541057aca0b18c5da`. It scanned the first 500,000 records, limiting each eligible candidate pool to 1,500. GPT supplied 1,285 candidates; Claude and Llama reached the cap. A seeded random sample selected 1,000 per family. This is sampling from capped pools in a source prefix, not uniform sampling from the complete dataset.

Selection and filtering

Only records with the selected model names and an English source language label were eligible. Collection used the first user/assistant pair; required prompt length 20-10,000 characters and response length 100-20,000; excluded redacted records, explicit moderation flags, email patterns, and broad model/provider-name patterns; and deduplicated conversation identifiers and normalized prompt-response pairs. Excluding explicit model names reduces an obvious attribution shortcut, although the broad pattern can also remove ordinary task content.

Preparation used seeded langdetect on prompt and response separately after removing fenced code and URLs. Text with fewer than 40 alphabetic characters was marked uncertain. English and uncertain examples were retained, so technical text was not automatically discarded. This removed 63 examples: 43 Claude, 10 GPT, and 10 Llama. Automatic detection remains imperfect.

Ethical handling and review status

The source access terms govern use and restrict dataset transfers [2]. Raw and processed conversations, including review previews, are excluded from GitHub. The repository publishes code, aggregate results, identifiers, and predictions without conversation text. Filters reduce obvious concerns but do not certify every record as free from private, sensitive, or inappropriate material. A training-data spot-check informed language/domain rule revisions; the saved preparation summary still records manual review as incomplete. No claim of a complete content audit is made.

Stored format

Local JSONL records include llm_name, llm_input, llm_output, sample_id, conversation_id, source, and source_revision. Preparation adds domain, group_id, and split. Collection/preparation summaries and a text-free split manifest document the process. Source records must be obtained independently under their terms.

2. Splits, domains, and leakage controls

Union-find connects examples that share an exact normalized prompt, exact normalized response, or conversation identifier. Normalization case-folds and collapses whitespace for duplicate detection, without changing stored text. Connections are transitive. There are 2,877 groups among 2,937 retained examples.

StratifiedGroupKFold creates ten shuffled folds with seed 42. Folds 0-1 form the test set, fold 2 validation, and folds 3-9 training. This approximately stratifies family labels while keeping connected examples together. It is one fixed split, not ten-fold model evaluation.

Split	Claude	GPT	Llama	Total
Training	670	693	693	2056
Validation	95	99	99	293
Test	192	198	198	588
Total	957	990	990	2937

Preparation asserts that grouped duplicates and normalized prompt/response keys do not cross split boundaries. Evaluation checks development/test group separation and neural training/validation file hashes. Vocabulary, TF-IDF weights, feature scaling, and classifiers are fitted on training data only. Validation selects checkpoints; main and additional training scripts do not load test data.

Preliminary domain assignment

Ordered rules use only prompts: summarization, transcript editing as writing, coding, prose writing, mathematics, then general. Coding takes precedence over ordinary writing keywords, so requests to write code remain coding. These rules are convenient proxies, not verified semantic labels; general is a fallback rather than a guaranteed factual-QA domain.

Domain	Claude	GPT	Llama	Total
Coding	119	99	107	325
General	777	825	791	2393
Math	8	16	8	32
Summarization	20	11	13	44
Writing	33	39	71	143

The writing test subset has 30 examples (11 Claude, 5 GPT, 14 Llama). Mathematics and summarization are too sparse for strong separate domain conclusions. The main test class majority is 198/588 = 33.67%; writing majority is 14/30 = 46.67%.

Remaining leakage and confounding risks

Exact matching cannot guarantee semantic near-duplicate removal. Grouping does not control for users, collection date, task difficulty, or model availability. Prompts were not deliberately matched across generators. Therefore above-chance input-only prediction may reflect which tasks users assigned to each model. A stronger future design would collect identical prompts across all generators and group the common prompts during splitting.

3. CNN/RNN implementation and training

A shared tokenizer preserves case, word/number tokens, punctuation, underscores, and newline markers. The training vocabulary is capped at 20,000 tokens with padding, unknown, separator, and newline special tokens. Input-only vocabulary contains 10,511 tokens. Unknown validation/test tokens map to the unknown token.

Component	TextCNN	Bidirectional LSTM
Embedding	Trainable, dimension 100	Trainable, dimension 100
Encoder	Conv1D widths 3/4/5; 64 filters each; ReLU	One LSTM layer; hidden size 64 per direction
Representation	Masked max pooling; concatenate 192 features	Concatenate final forward/backward states: 128 features
Padding handling	Mask padded convolution windows; one padded window for very short text	Packed sequences with true lengths
Output	Dropout 0.4; linear 192 to 3	Dropout 0.4; linear 128 to 3
Parameters, output/both	2,077,571	2,085,379

Input mode uses at most 127 prompt tokens; output mode uses at most 256 response tokens. Combined mode concatenates the same prompt and response budgets with a separator, up to 384 tokens. Thus combined experiments do not replace response tokens with prompt tokens, although the longer sequence changes the learning problem.

Training setting	Value
Environment	Windows; Python 3.11.9; PyTorch 2.14.1+cpu; 4 CPU threads
Optimizer / loss	Adam, learning rate 0.001 / cross-entropy
Batch size / maximum epochs	32 / 15
Regularization	Dropout 0.4; gradient norm clipping at 1.0
Selection / stopping	Best validation macro-F1; patience 3; seed 42

CNN learns local token patterns and pools their strongest activations. The LSTM processes order-dependent context in both directions. Neither model uses pretrained embeddings. logits enter cross-entropy directly; argmax gives class predictions. Saved model smoke checks verify output shapes, finite values, backward gradients, and prediction invariance to extra padding.

Baseline and evaluation measures

TF-IDF uses the same token budgets and case handling, unigram/bigram features, minimum document frequency 2, sublinear term frequency, and at most 20,000 features. Logistic regression uses C = 1 and max_iter = 2,000. Accuracy is correct predictions divided by examples. Each class F1 is 2TP/(2TP+FP+FN); macro-F1 is the unweighted mean over the three classes. Per-class precision/recall and confusion matrices expose uneven performance.

4. Main results: RQ1 and RQ2

Model	Text mode	Test accuracy	Test macro-F1
CNN	Input	42.52%	0.4231
CNN	Output	91.33%	0.9137
CNN	Input + output	90.65%	0.9062
BiLSTM	Input	40.14%	0.4005
BiLSTM	Output	85.20%	0.8518
BiLSTM	Input + output	72.28%	0.7255
TF-IDF + LR	Input	41.50%	0.4132
TF-IDF + LR	Output	87.93%	0.8791
TF-IDF + LR	Input + output	87.93%	0.8795

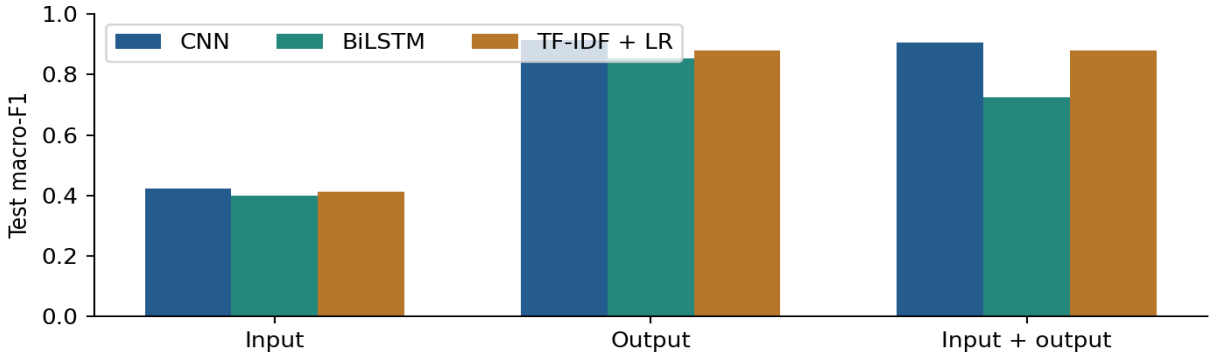


Figure 1. All main models use the same 588 test examples. Values are from saved final evaluation predictions; macro-F1 is shown on a 0-1 scale.

RQ1: Can response text identify the generator?

Yes, within the sampled three-class setting. CNN output-only correctly classifies 537/588 examples (91.33%, macro-F1 0.9137). BiLSTM classifies 501/588 (85.20%, macro-F1 0.8518), and the baseline achieves 87.93%. The CNN exceeds the LSTM by 6.12 accuracy percentage points. This is an observed single-run comparison, not a statistical-significance claim or a universal ranking of architectures.

RQ2: Does including the prompt help?

Input-only accuracy is 42.52% for CNN, 40.14% for BiLSTM, and 41.50% for TF-IDF, above 33.33% uniform-random expected accuracy. Prompt-selection and task-mix differences are plausible explanations; a prompt cannot reveal the generator by necessity. Adding input reduces CNN accuracy by 0.68 percentage points and BiLSTM by 12.93 points relative to output-only. TF-IDF accuracy ties; its macro-F1 changes slightly. Longer sequences, irrelevant prompt content, and limited training data may contribute to the LSTM decrease, but the experiments do not isolate these mechanisms.

5. Errors, training behavior, and model selection

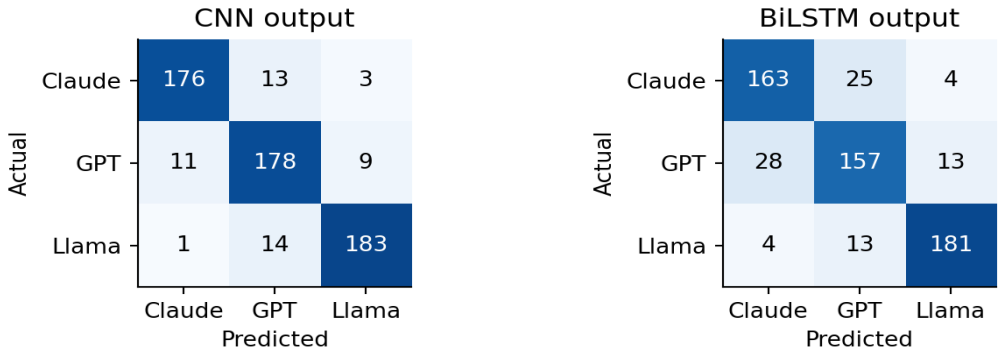


Figure 2. Response-only confusion matrices. Rows are true families; columns are predictions. Both models find GPT the hardest class by F1.

Model	Class	Precision	Recall	F1	Support
CNN	Claude	0.936	0.917	0.926	192
CNN	GPT	0.868	0.899	0.883	198
CNN	Llama	0.938	0.924	0.931	198
BiLSTM	Claude	0.836	0.849	0.842	192
BiLSTM	GPT	0.805	0.793	0.799	198
BiLSTM	Llama	0.914	0.914	0.914	198

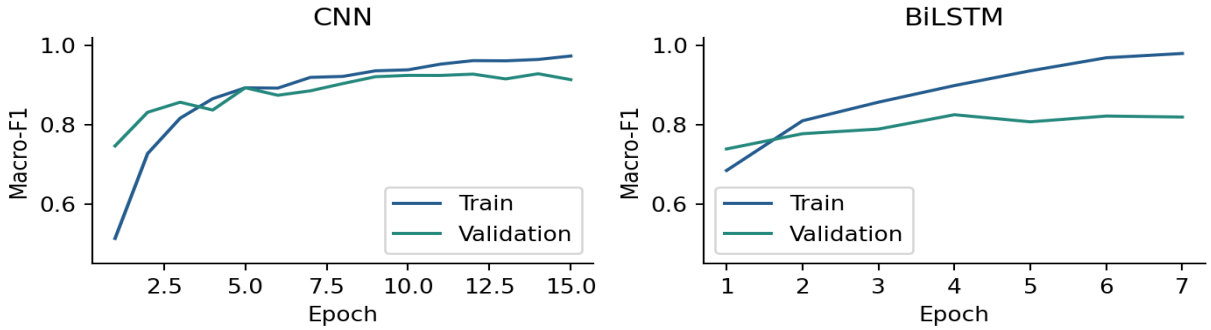


Figure 3. Response-only training and validation macro-F1 across completed epochs. Early stopping preserves the selected checkpoint rather than the final epoch.

CNN output-only selects epoch 14 (validation macro-F1 0.9290); BiLSTM selects epoch 4 (0.8256) and stops after epoch 7. The LSTM training F1 rises to 0.9801 while validation F1 is 0.8198 at epoch 7, indicating overfitting. Input-only runs also fit training text strongly while validation performance remains much lower. High training performance therefore does not establish generalization.

The six main best epochs are CNN input/output/both: 7/14/9, and BiLSTM: 9/4/8. Test evaluation follows checkpoint selection. Results are preserved rather than using test outcomes to select new hyperparameters. CPU timings vary with workload, so logged epoch durations are not controlled speed benchmarks.

6. Cross-domain transfer: RQ3

Two additional response-only models exclude writing-labeled groups from training and validation. The remaining training set has 1,960 examples (Claude 649, GPT 664, Llama 647), and validation has 276 (94, 94, 88). The model architecture, optimizer, token budgets, seed, and stopping rule remain the same; selection uses non-writing validation macro-F1. No writing test examples enter training or selection.

The additional CNN selects epoch 8 (non-writing validation macro-F1 0.9127); the LSTM selects epoch 6 (0.8405). Original and writing-excluded models are compared on identical writing/non-writing test subsets.

Model / training	Test domain	N	Accuracy	Macro-F1
CNN / original	Writing	30	86.67%	0.8456
CNN / original	Non-writing	558	91.58%	0.9165
CNN / excluded	Writing	30	76.67%	0.7388
CNN / excluded	Non-writing	558	89.25%	0.8923
BiLSTM / original	Writing	30	86.67%	0.8489
BiLSTM / original	Non-writing	558	85.13%	0.8515
BiLSTM / excluded	Writing	30	83.33%	0.7827
BiLSTM / excluded	Non-writing	558	80.65%	0.8075

Observed transfer and comparison

On writing, the excluded-domain CNN achieves 76.67% accuracy and macro-F1 0.7388, compared with 86.67% and 0.8456 for the original CNN. The excluded LSTM achieves 83.33% and 0.7827, compared with 86.67% and 0.8489. These correspond to 23/30 versus 26/30 correct for CNN and 25/30 versus 26/30 for LSTM. A single writing example changes accuracy by 3.33 percentage points.

Excluded-domain models retain useful attribution signals on writing, consistent with some transferable patterns. However, the 30-example sample cannot establish that fingerprints generalize broadly. Original models also saw writing during development, so their writing performance is a reference rather than an unseen-domain evaluation. Lower non-writing performance after exclusion shows that differences are not confined to writing.

Do some generators transfer more consistently?

Per-family recall for writing-excluded models is shown below. Different supports make class comparisons unstable, particularly GPT with only five examples. Neither table supports a broad family-level generalization claim.

Excluded model	Claude recall (n=11)	GPT recall (n=5)	Llama recall (n=14)
CNN	0.727	0.600	0.857
BiLSTM	1.000	0.600	0.786

Removing writing reduces training size and changes learned vocabularies and stochastic optimization. These effects are confounded with domain removal. Heuristic overlap between task categories also remains possible. A larger manually labeled domain test, several seeds, and matched-size training controls would strengthen RQ3.

7. Linguistic and structural signals: RQ4

Descriptive statistics use training responses only. Features include characters, words, mean word length, lexical diversity among the first up to 100 words, newline density, bullet/numbered lines, code fences, question/exclamation density, and punctuation fraction. These features can reflect task mix as well as generator style.

Family	Words	First-100 TTR	Newlines / 1,000 chars	Bullet lines
Claude	190.2	0.625	18.00	2.52
GPT	159.1	0.690	10.86	0.55
Llama	182.4	0.665	8.85	0.42

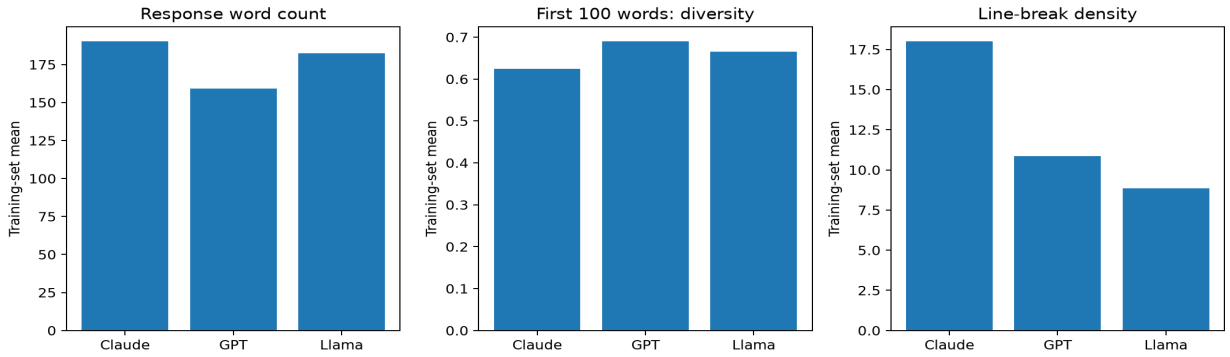


Figure 4. Training-set feature means. TTR is distinct words divided by words in the first up to 100-word span. For shorter responses it remains length-sensitive.

Claude has the largest mean word count and newline/bullet frequency in this sample. GPT has the highest first-span lexical diversity. Llama has greater exclamation density in the full feature table. These are sample associations, not permanent properties of the families.

Controlled feature comparisons

Experiment	Test accuracy	Test macro-F1
Original output TF-IDF	87.93%	0.8791
Without punctuation / newlines	87.24%	0.8726
Style features only	63.61%	0.6340

The style-only classifier standardizes numerical features using training statistics and fits logistic regression. Accuracy of 63.61% indicates that structural and length-related features alone contain predictive information. The lexical ablation removes punctuation, underscores, and newline markers after selecting the original first 256 response tokens; TF-IDF/logistic regression is retrained on those features.

Formatting removal lowers accuracy by 0.68 percentage points (87.93% to 87.24%), leaving substantial predictive performance. Lexical features appear useful beyond formatting, but ablation changes the feature space and creates new word adjacencies. Style-only uses full responses while TF-IDF uses truncated text. This analysis does not directly identify what CNN/LSTM neurons use; attribution or neural retraining ablations would be additional experiments.

8. Lessons, limitations, and reproducibility

Lessons and experience from the project

Data preparation changes the experiment. Source English labels did not eliminate all language concerns. A second language check removed 63 examples, with an uneven effect across families. Prompt review also exposed weaknesses in broad domain keywords. These issues show why data inspection and transparent filtering matter before model training.

Preventing leakage requires a relationship-based split. A row-level random split could place the same prompt or response in development and test. Connecting duplicates before splitting preserves that relationship. Exact grouping is a useful safeguard but does not solve semantic overlap or prompt-selection bias.

Validation performance matters more than fitting training text. The LSTM response model continued improving on training while validation stagnated. Early stopping retained epoch 4. The prompt-only control further showed that seemingly useful performance can come from dataset associations, not response style alone.

A simple baseline is informative. TF-IDF logistic regression exceeded response-only LSTM accuracy and approached CNN performance. More complex architecture did not guarantee stronger held-out performance. Comparisons against a shared-budget baseline helped interpret the neural results.

Small analyses require cautious interpretation. Thirty writing test examples and a small formatting-ablation change support exploratory findings, not strong significance claims. Full-response style features and truncated lexical features also need their scope stated explicitly.

Reproducibility includes the working environment. Development used the virtual environment Python executable when the Windows py launcher was unavailable. Saved configurations, histories, hashes, and a pinned source revision document the run. Git exclusions separate publishable results from local conversation files and checkpoints. LLM assistance supported development; understanding and verification remain the author's responsibility.

Practical limits and future work

The findings concern one version per family, one source, one split, and one seed. Source-prefix sampling, unequal task distributions, heuristic domains, imperfect language/identity filtering, and incomplete manual review limit representativeness. Future work should use common prompts across generators, larger verified domain holdouts, matched-size cross-domain controls, several seeds, semantic duplicate checks, and neural ablations. Test data should remain reserved for final evaluation of any newly designed experiments.

Reproduction and references

The repository README lists installation and run order: collection, preparation, model smoke checks, six main neural runs, baseline, two writing-excluded runs, style analysis, and final evaluation. Checkpoints and conversations are excluded and must be regenerated locally. Existing result directories must be backed up before a fresh run. The package freeze records the development environment; exact CPU timing is machine-dependent.

[1] Zheng, L., Chiang, W.-L., Sheng, Y., et al. (2023). *LMSYS-Chat-1M: A Large-Scale Real-World LLM Conversation Dataset.* arXiv:2309.11998. https://arxiv.org/abs/2309.11998

[2] LMSYS. *LMSYS-Chat-1M dataset card and license agreement.* Accessed October 11, 2026. https://huggingface.co/datasets/lmsys/lmsys-chat-1m